



UFRJ



INSTITUTO DE
COMPUTAÇÃO
UFRJ

Programação de Computadores II

Métodos de Busca e Complexidade de Algoritmos

Prof. Marcos Tomazzoli Leipnitz

Sumário

- Busca Sequencial
- Busca Binária
- Introdução à Complexidade de Algoritmos

Busca em Arranjo

- Suponha que temos um arranjo de inteiros
- Como fazemos para verificar se um determinado número está lá?

Busca Sequencial

- Varremos o arranjo, da esquerda para a direita
- Se acharmos o número, então ele está no arranjo
- Se chegarmos ao final do arranjo e não acharmos, ele não está

Busca Sequencial

- Implementação em C

```
int buscaSeq(int arr[], int tam,
int el) {
    int i;
    for (i=0; i<tam; i++)
        if (arr[i] == el) return i;
    return -1; }
```

Busca Sequencial - Implementação

- Função já dispõe, em seus parâmetros, de toda a informação de que precisa para executar
 - Do arranjo propriamente dito (**arr**), de seu tamanho (**tam**) e do valor que está sendo buscado (**el**)

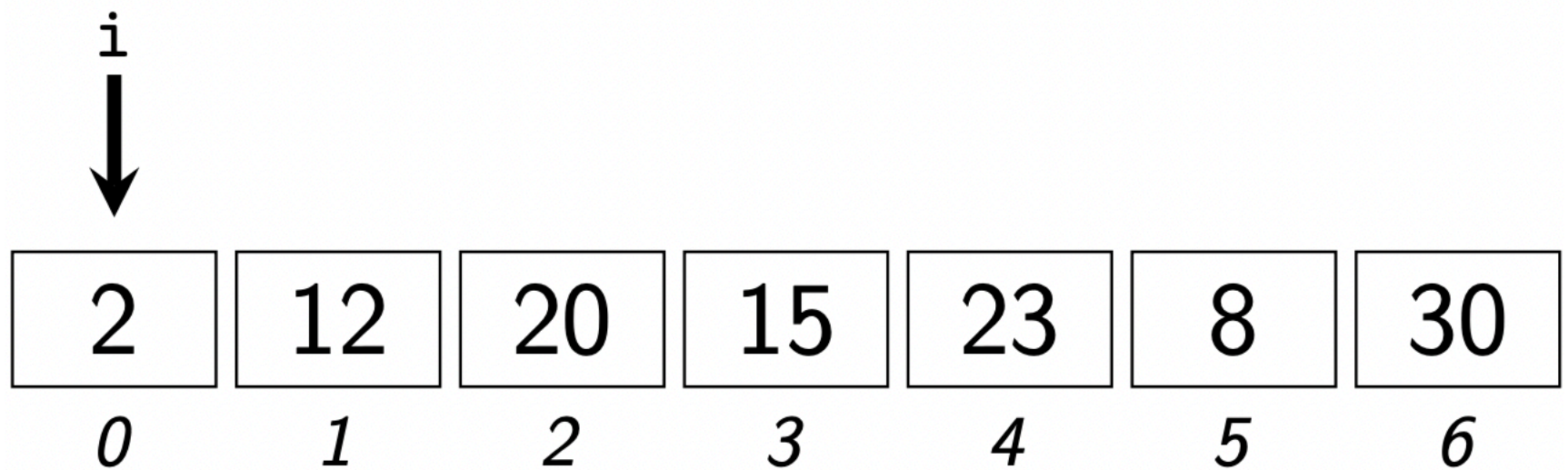
Busca Sequencial - Exemplo

- Buscando 15

2	12	20	15	23	8	30
<i>0</i>	<i>1</i>	<i>2</i>	<i>3</i>	<i>4</i>	<i>5</i>	<i>6</i>

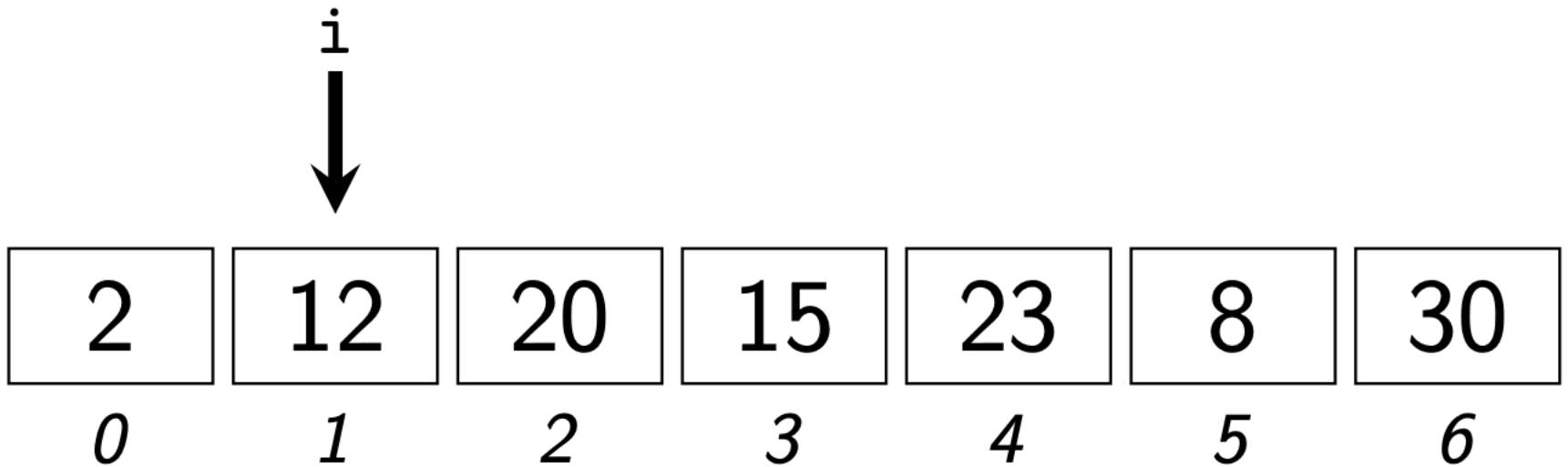
Busca Sequencial - Exemplo

- Buscando 15



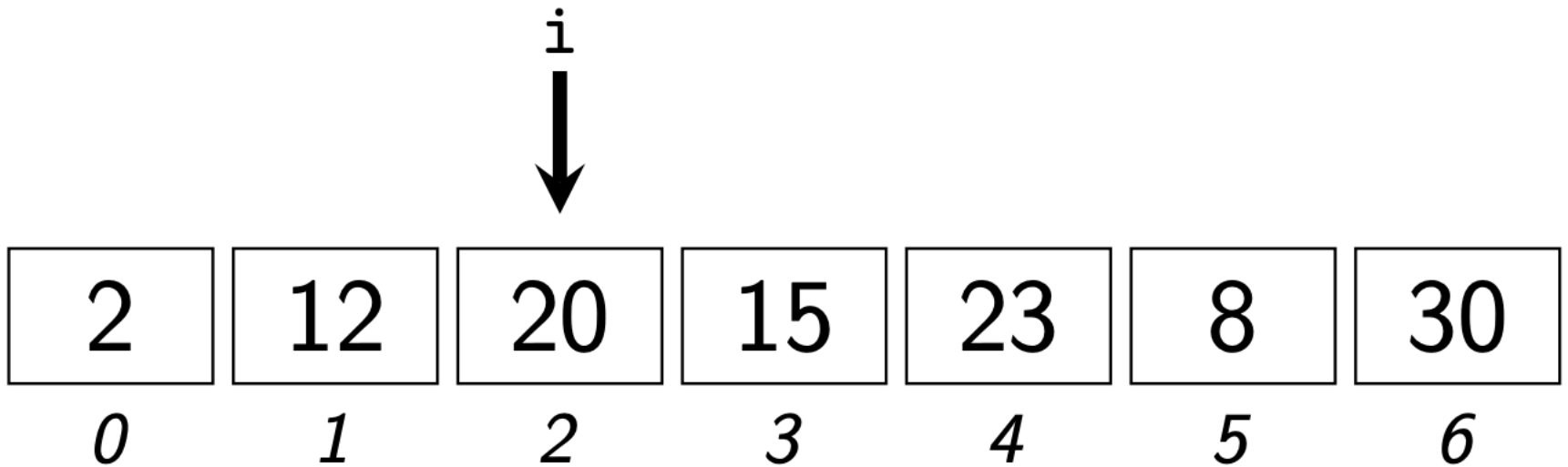
Busca Sequencial - Exemplo

- Buscando 15



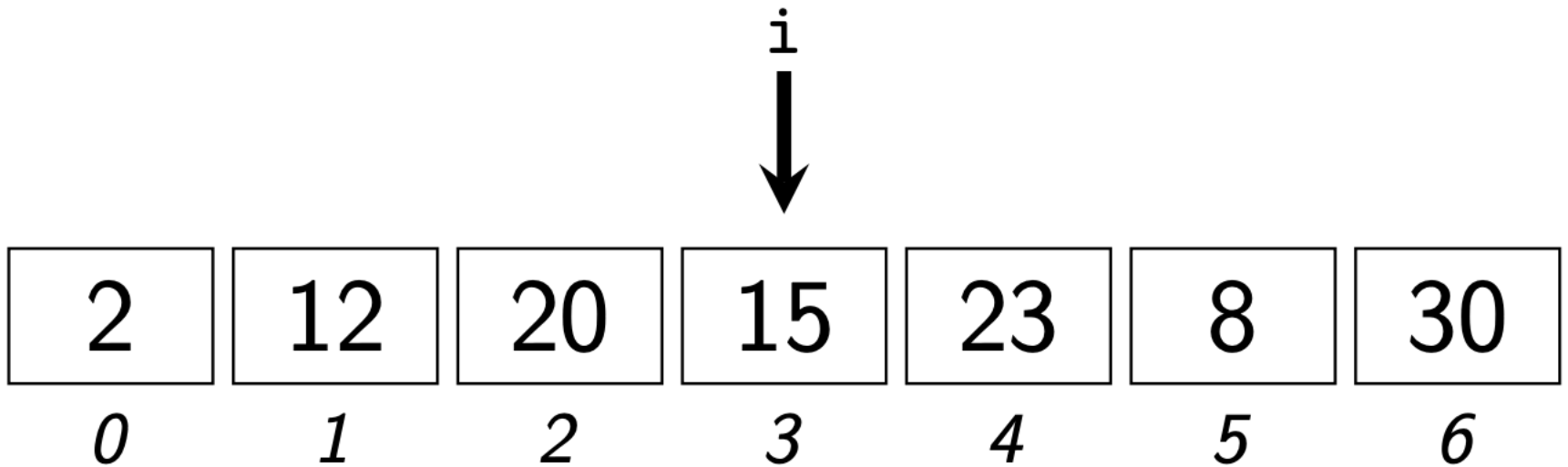
Busca Sequencial - Exemplo

- Buscando 15



Busca Sequencial - Exemplo

- Buscando 15



Busca Sequencial - Exemplo

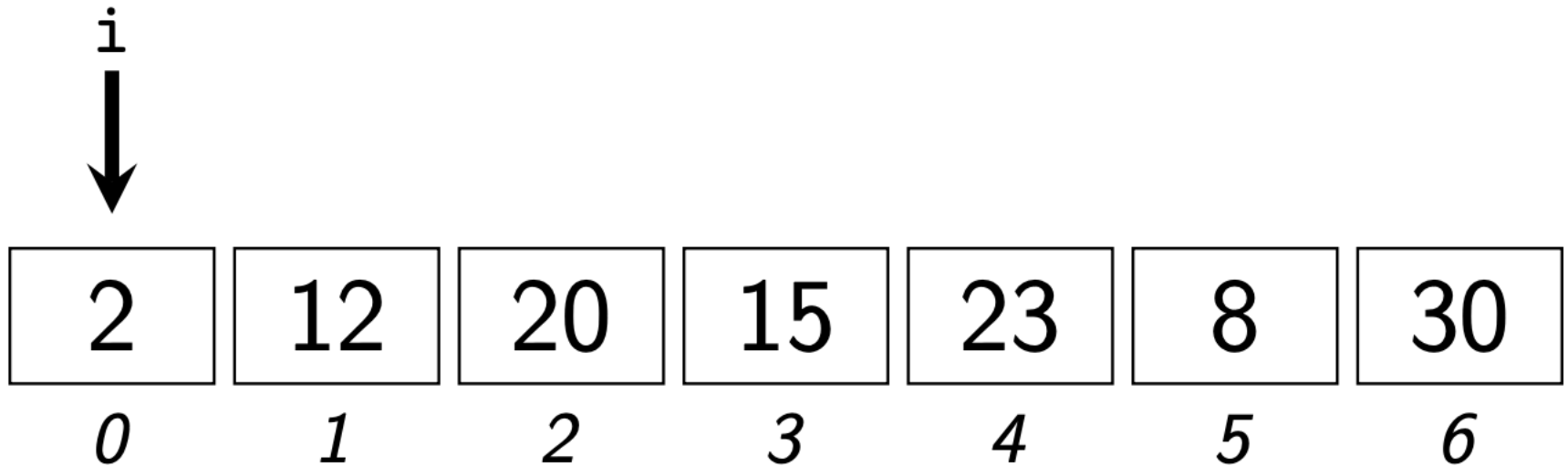
- Buscando 15

2	12	20	15	23	8	30
<i>0</i>	<i>1</i>	<i>2</i>	<i>3</i>	<i>4</i>	<i>5</i>	<i>6</i>

Encontrou! Índice 3 (quarta posição).

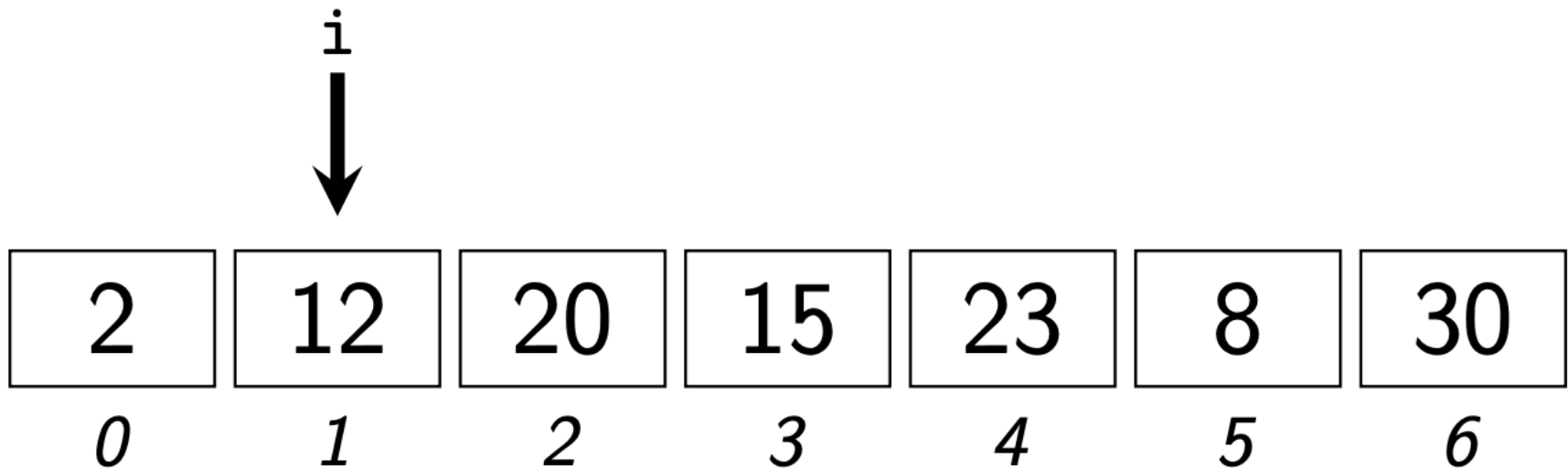
Busca Sequencial - Exemplo

- Agora buscando 16



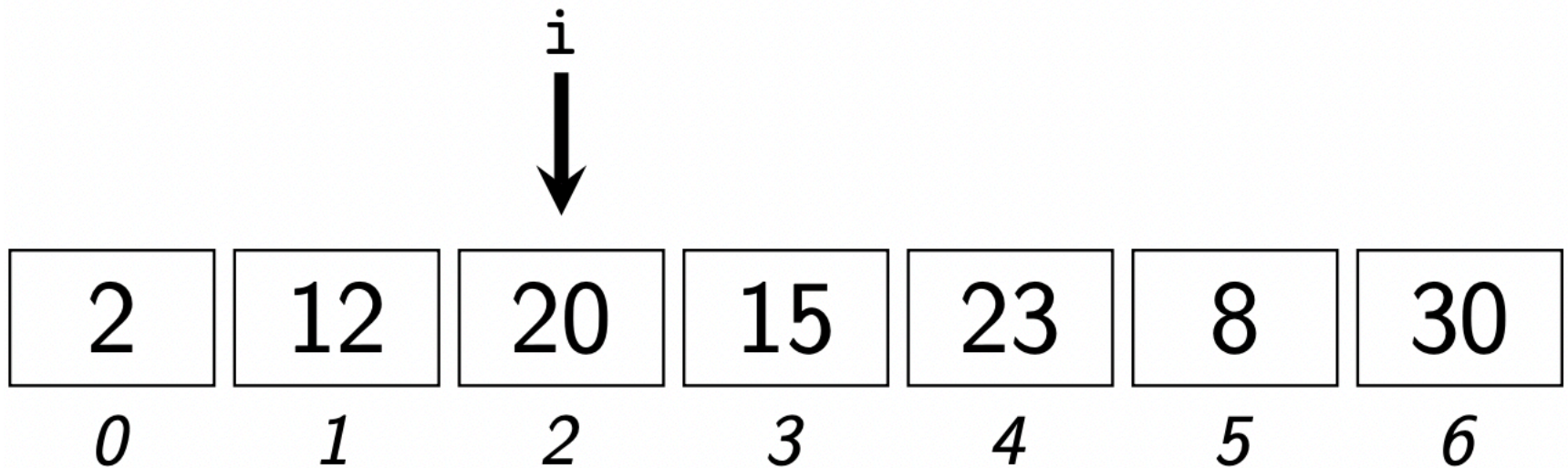
Busca Sequencial - Exemplo

- Agora buscando 16



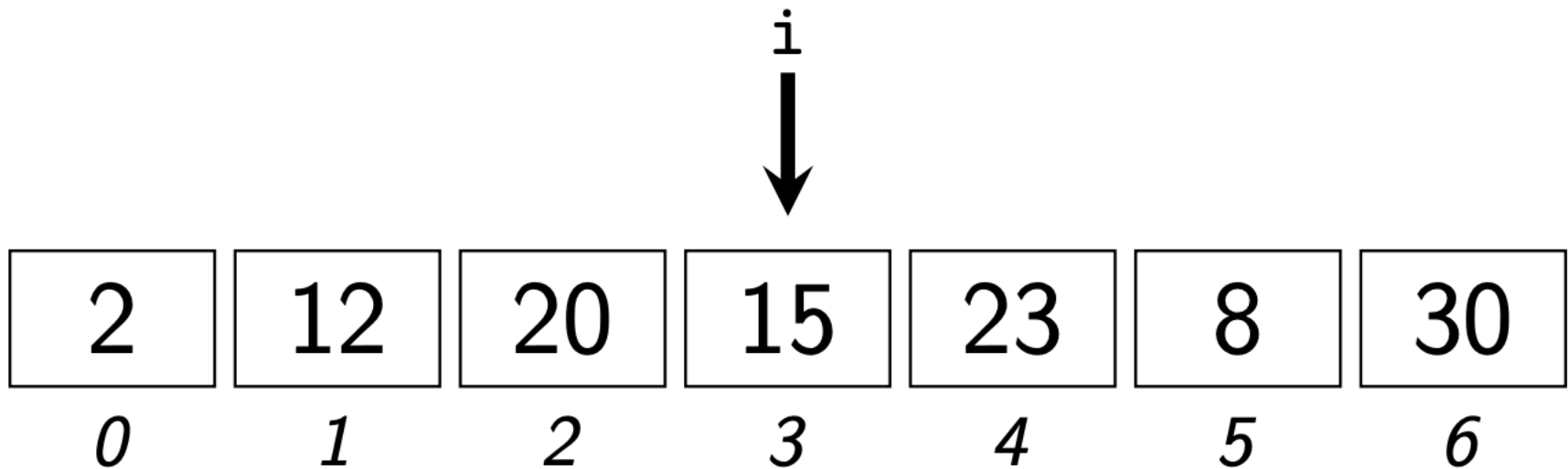
Busca Sequencial - Exemplo

- Agora buscando 16



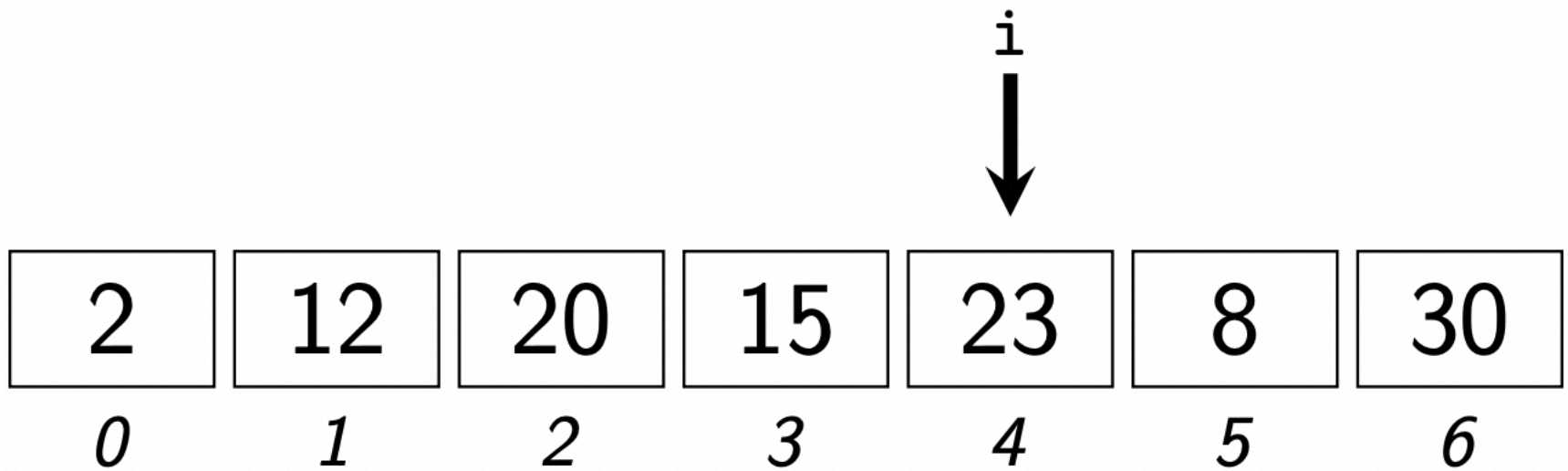
Busca Sequencial - Exemplo

- Agora buscando 16



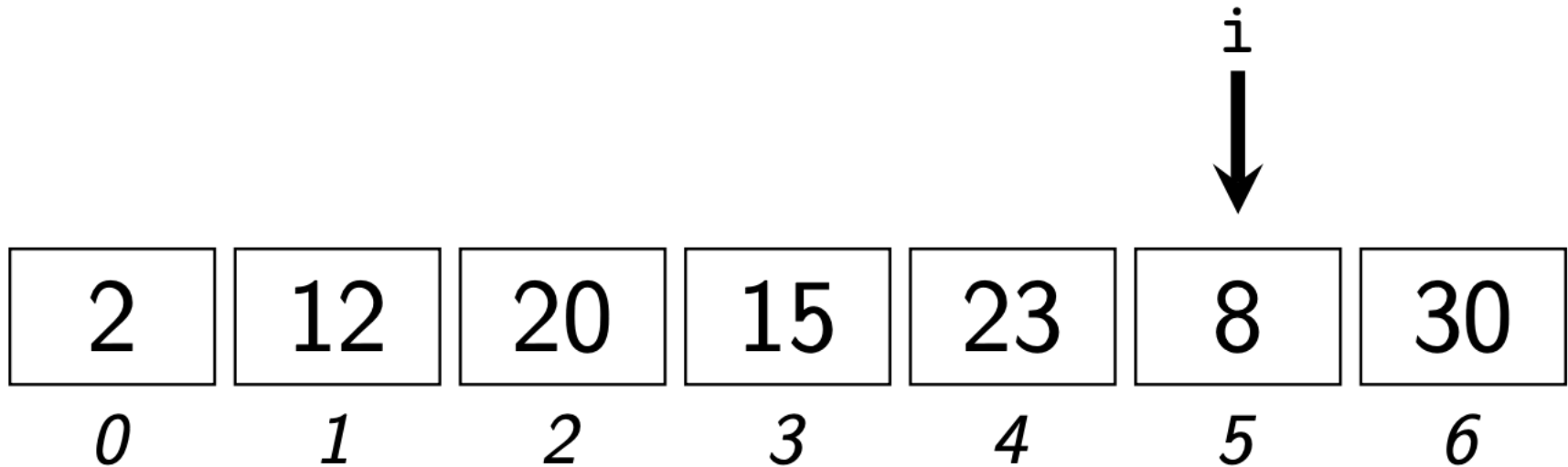
Busca Sequencial - Exemplo

- Agora buscando 16



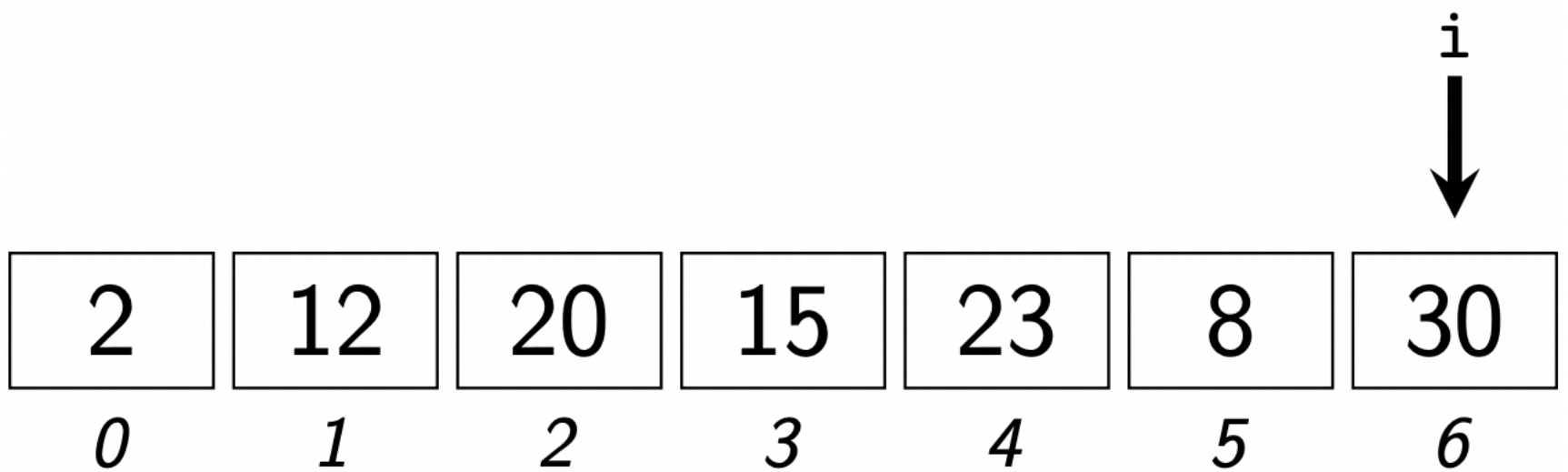
Busca Sequencial - Exemplo

- Agora buscando 16



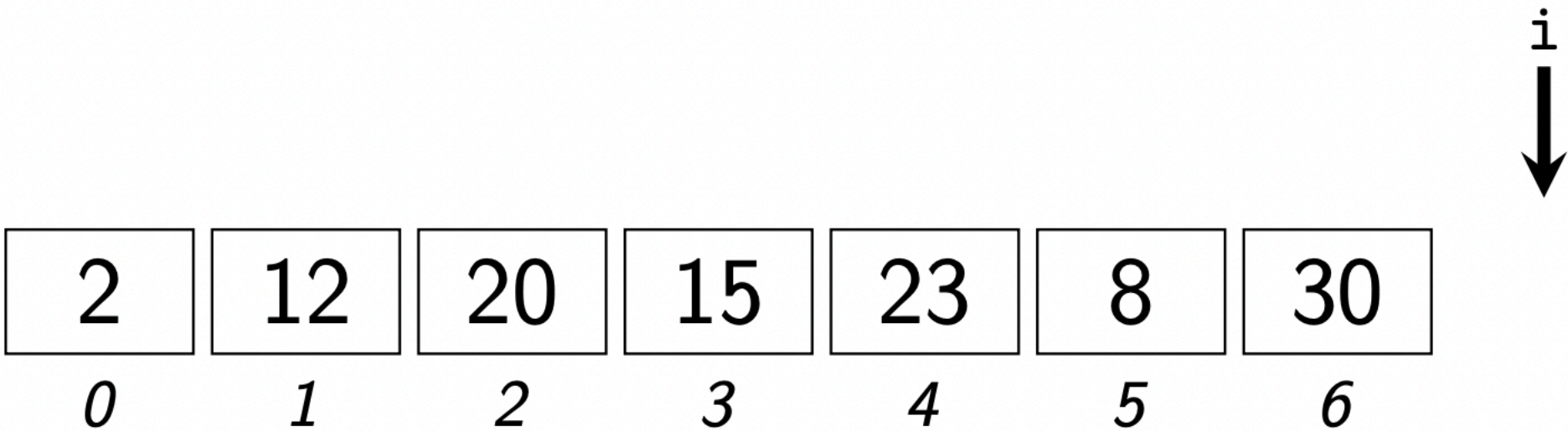
Busca Sequencial - Exemplo

- Agora buscando 16



Busca Sequencial - Exemplo

- Agora buscando 16



Não encontrou! $i == 7$, ou seja, $i == \text{tamanho}$, retorna -1

Busca Sequencial

- O que podemos fazer para melhorar a busca?
 - Nada, realmente...
- E se o arranjo estivesse ordenado?
 - Poderíamos parar a busca assim que encontrássemos um número maior que o sendo buscado

Busca Sequencial - Arranjo Ordenado

- Implementação em C

```
int buscaSeqOrd(int arr[], int tam,
int el) {
    int i;
    for (i=0; i<tam; i++) {
        if (arr[i] >= el) {
            if (arr[i] == el) return i;
            else return -1; }
    }
    return -1; }
```

Busca Sequencial - Arranjo Ordenado

- Com o arranjo ordenado, potencialmente executamos menos comparações no caso do elemento não estar no arranjo
- Estar ordenado também torna fácil algumas tarefas:
 - Busca pelo menor elemento: $v[0]$
 - Busca pelo maior elemento:
 $v[tamanho-1]$

Busca Sequencial - Arranjo Ordenado

- Vimos que se o arranjo estiver ordenado, buscas ficam em geral mais rápidas
- Paramos a busca assim que uma das condições for satisfeita:
 - Encontramos o elemento buscado
 - Chegamos ao fim do arranjo
 - (Diferencial!) Encontramos um elemento maior que o buscado

Busca Sequencial - Arranjo Ordenado

- Ainda assim, no **pior caso**, teremos que olhar o arranjo inteiro, quando:
 - O elemento buscado for o último
 - O elemento buscado não estiver no arranjo, mas for maior que o último
- Não teria um modo melhor?

Busca Binária

- Verifique se o elemento buscado é o do meio do arranjo
- Se não for, verifique se é maior
 - Se for, repita a busca na metade direita do arranjo
 - Se não for, repita a busca na metade esquerda do arranjo

Busca Binária

- Implementação em C

```
int buscaBin(int arr[], int tam, int el){
    int fim = tam-1;
    int ini = 0;
    while (ini <= fim) {
        int meio = (fim + ini)/2;
        if (arr[meio] < el)
            ini = meio + 1;
        else {
            if (arr[meio] > el)
                fim = meio - 1;
            else return meio;
        }
    }
    return -1; }
```

Busca Binária - Implementação

- Note que retornamos:
 - O índice no arranjo do elemento buscado
 - Ou -1 em caso de erro

Busca Binária - Exemplo

- Buscando 52

0									9
-78	-4	0	32	52	55	63	69	125	200

Busca Binária - Exemplo

- Buscando 52

0										9
-78	-4	0	32	52	55	63	69	125	200	
↑									↑	
i									f	

Busca Binária - Exemplo

- Buscando 52

0				4					9
-78	-4	0	32	52	55	63	69	125	200
↑				↑					↑
i				m					f

Busca Binária - Exemplo

- Buscando 52

0				4					9
-78	-4	0	32	52	55	63	69	125	200
↑				↑					↑
i				m					f

$v[m] == 52?$

Busca Binária - Exemplo

- Buscando 52

0				4					9
-78	-4	0	32	52	55	63	69	125	200
↑				↑					↑
i				m					f

$v[m] == 52?$ Achou com apenas 1 acesso

Busca Binária - Exemplo

- Buscando 55

0									9
-78	-4	0	32	52	55	63	69	125	200

Busca Binária - Exemplo

- Buscando 55

0										9
-78	-4	0	32	52	55	63	69	125	200	
↑									↑	
i									f	

Busca Binária - Exemplo

- Buscando 55

0				4					9
-78	-4	0	32	52	55	63	69	125	200
↑				↑					↑
i				m					f

Busca Binária - Exemplo

- Buscando 55

0				4					9
-78	-4	0	32	52	55	63	69	125	200
↑				↑					↑
i				m					f

$v[m] == 55?$

Busca Binária - Exemplo

- Buscando 55

0				4					9
-78	-4	0	32	52	55	63	69	125	200
↑				↑					↑
i				m					f

$v[m] == 55?$ Não. $v[m] < 55?$

Busca Binária - Exemplo

- Buscando 55

0				4					9
78	4	0	32	52	55	63	69	125	200
↑				↑					↑
i				m					f

$v[m] == 55?$ Não. $v[m] < 55?$ Sim.

Busca Binária - Exemplo

- Buscando 55

0				4					9
7	8	4	0	32	55	63	69	125	200
				↑	↑				↑
				m	i				f

Busca Binária - Exemplo

- Buscando 55

0				4					9
7 / 8	4	0	3 / 2	5 / 2	55	63	69	125	200
					↑		↑		↑
					i		m		f

Busca Binária - Exemplo

- Buscando 55

0				4					9
7	8	4	0	3	55	63	69	125	200
					↑		↑		↑
					i		m		f
$v[m] == 55?$									

Busca Binária - Exemplo

- Buscando 55

0				4					9
7	8	4	0	3	55	63	69	125	200
					↑		↑		↑
					i		m		f

$v[m] == 55?$ Não. $v[m] < 55?$

Busca Binária - Exemplo

- Buscando 55

0				4					9	
7	8	4	0	3	2	55	63	69	125	200
						↑		↑		↑
						i		m		f

$v[m] == 55?$ Não. $v[m] < 55?$ Não.

Busca Binária - Exemplo

- Buscando 55

0				4					9						
7	8	4	0	3	2	55	63	6	9	1	2	5	2	0	0
						↑			↑					↑	
						i			m					f	

Busca Binária - Exemplo

- Buscando 55

0				4					9						
7	8	4	0	3	2	55	63	6	9	1	2	5	2	0	0
						↑	↑	↑							
						i	f	m							

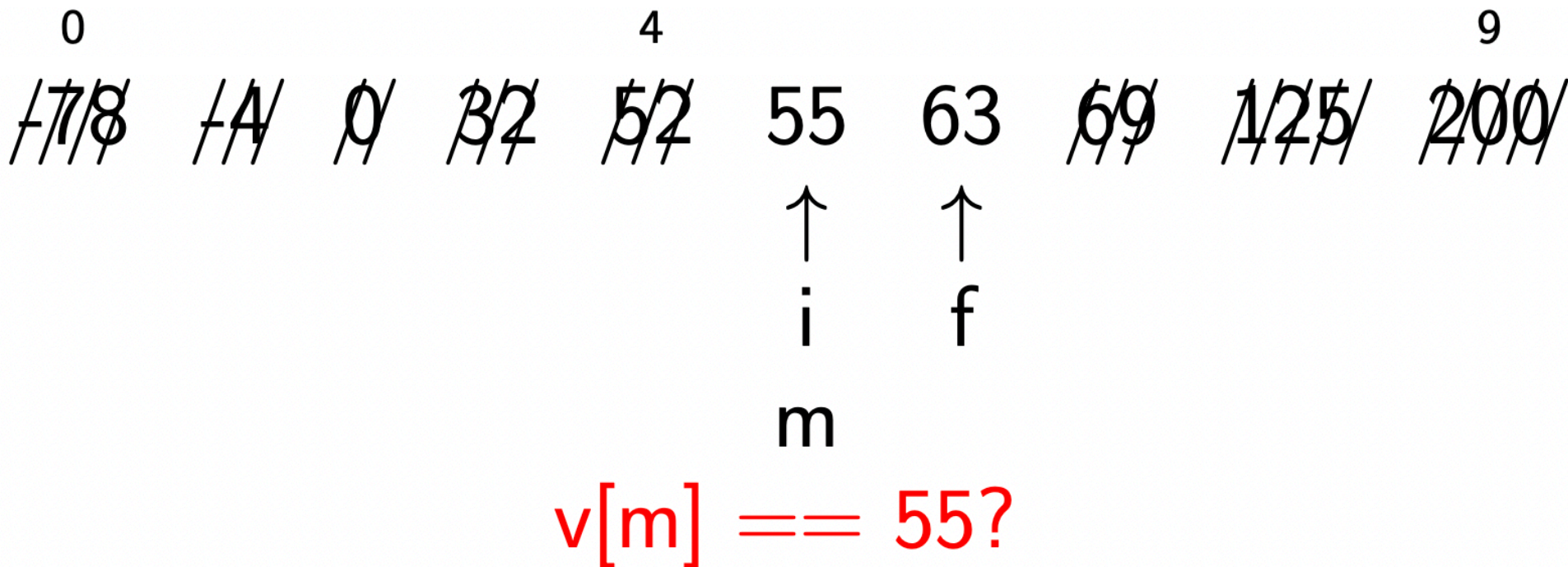
Busca Binária - Exemplo

- Buscando 55

0				4					9	
7	8	4	0	3	2	55	63	69	125	200
						↑	↑			
						i	f			
						m				

Busca Binária - Exemplo

- Buscando 55



Busca Binária - Exemplo

- Buscando 55

0				4					9
7/8	4	0	3/2	5/2	55	63	69	125	200
					↑	↑			
					i	f			
					m				

$v[m] == 55?$ Sim

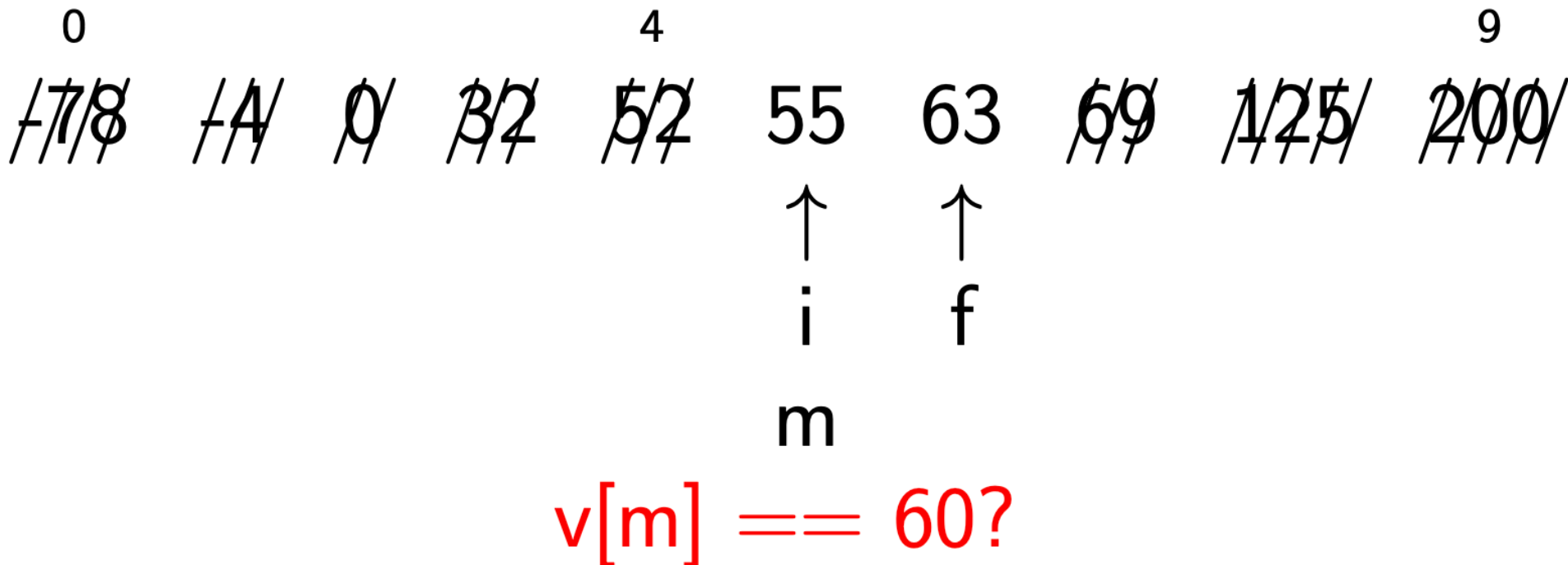
Busca Binária - Exemplo

- Buscando 60

0				4					9
7/8	4	0	3/2	5/2	55	63	69	125	200
					↑	↑			
					i	f			
					m				

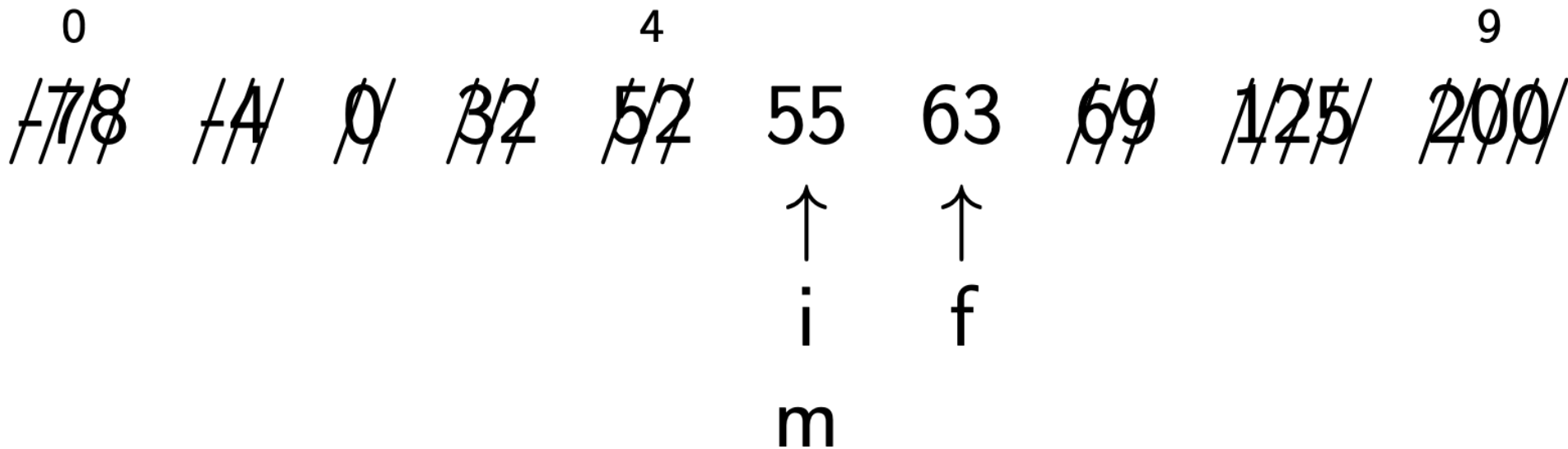
Busca Binária - Exemplo

- Buscando 60



Busca Binária - Exemplo

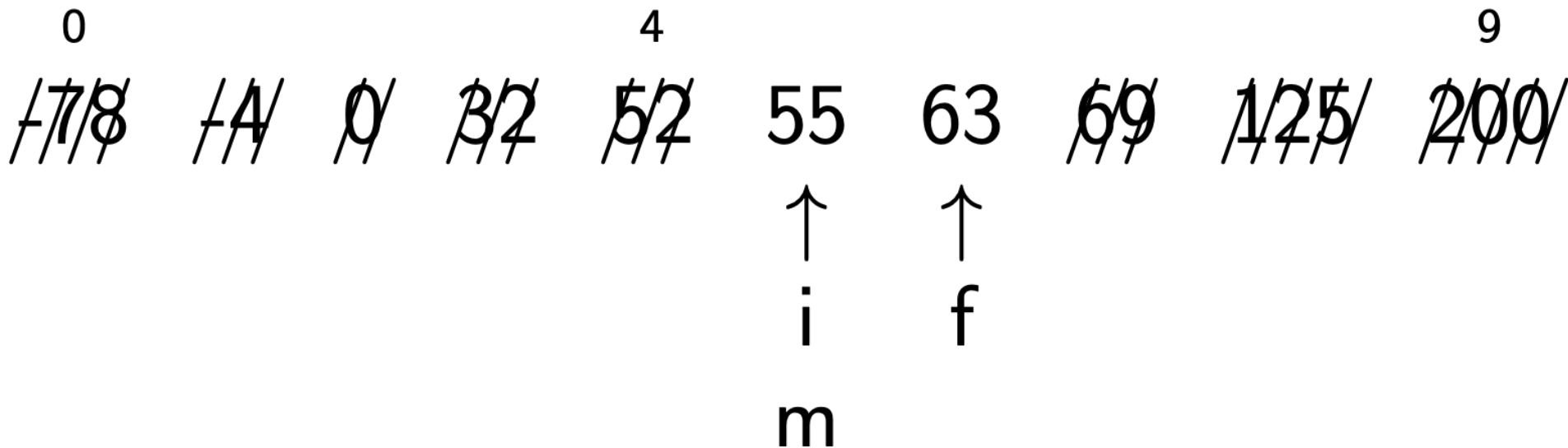
- Buscando 60



$v[m] == 60?$ Não. $v[m] < 60?$

Busca Binária - Exemplo

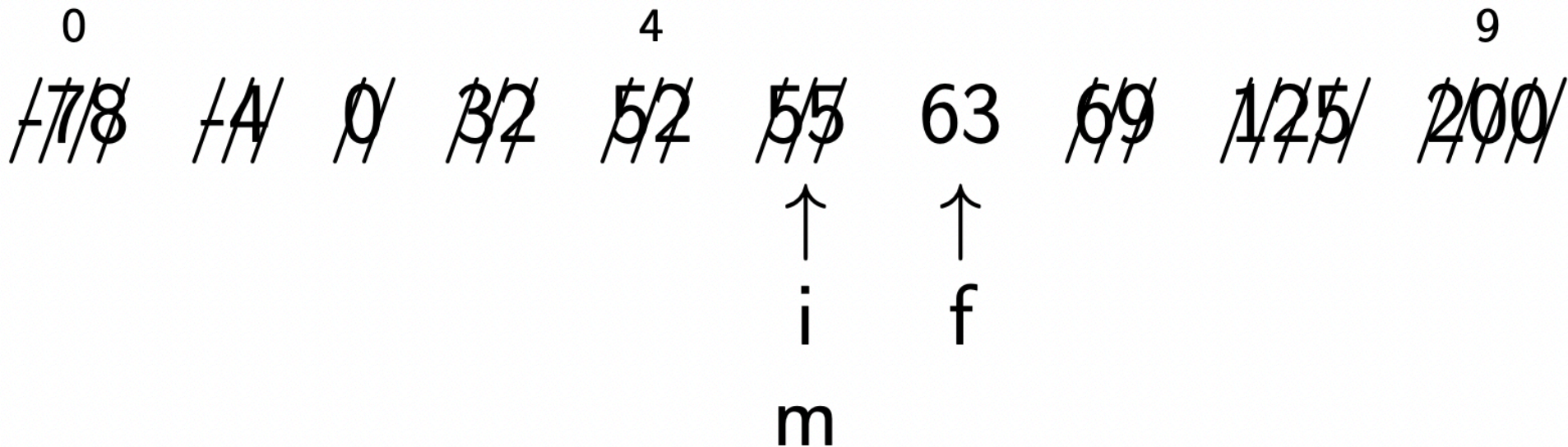
- Buscando 60



$v[m] == 60?$ Não. $v[m] < 60?$ Sim

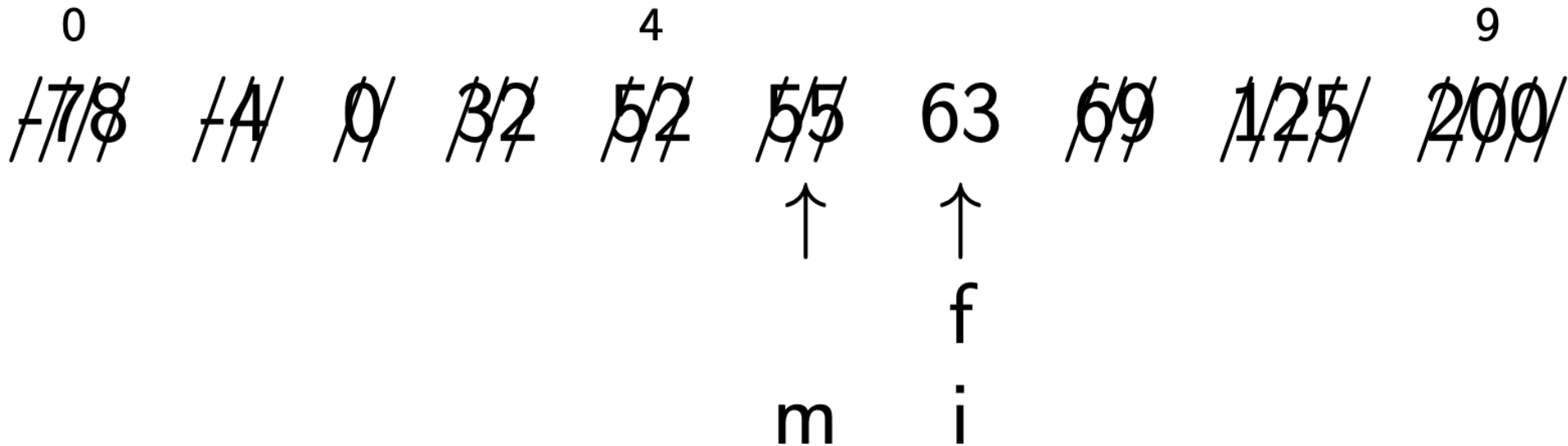
Busca Binária - Exemplo

- Buscando 60



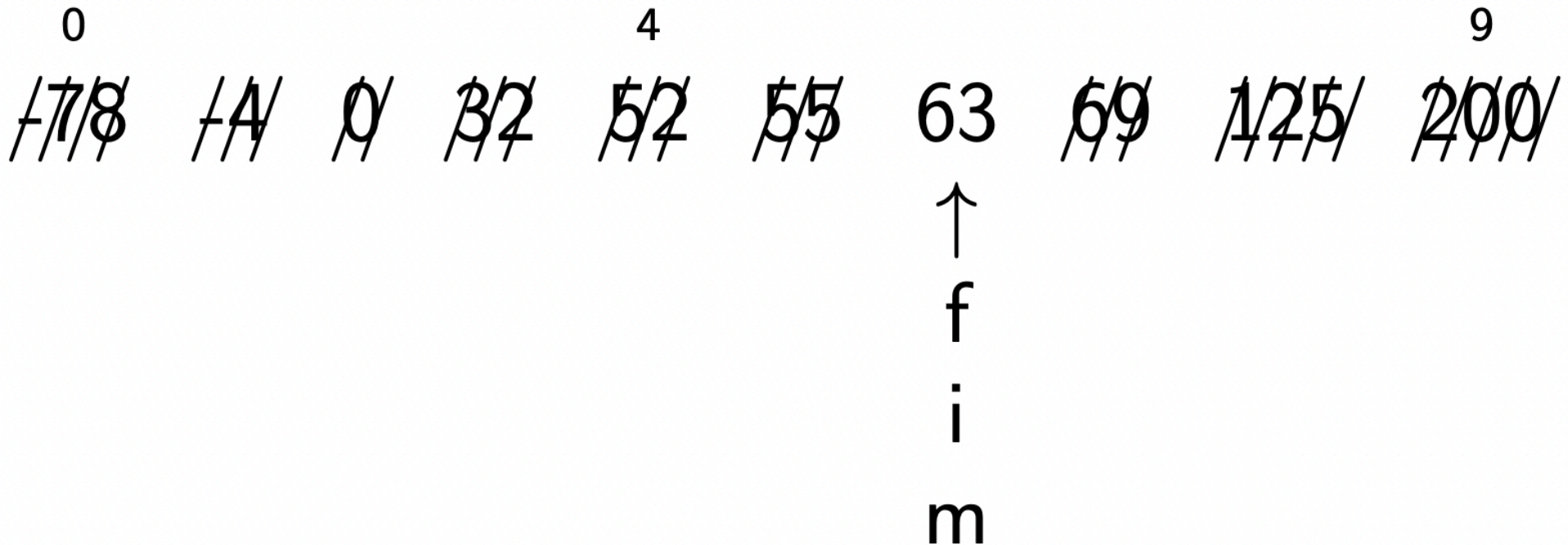
Busca Binária - Exemplo

- Buscando 60



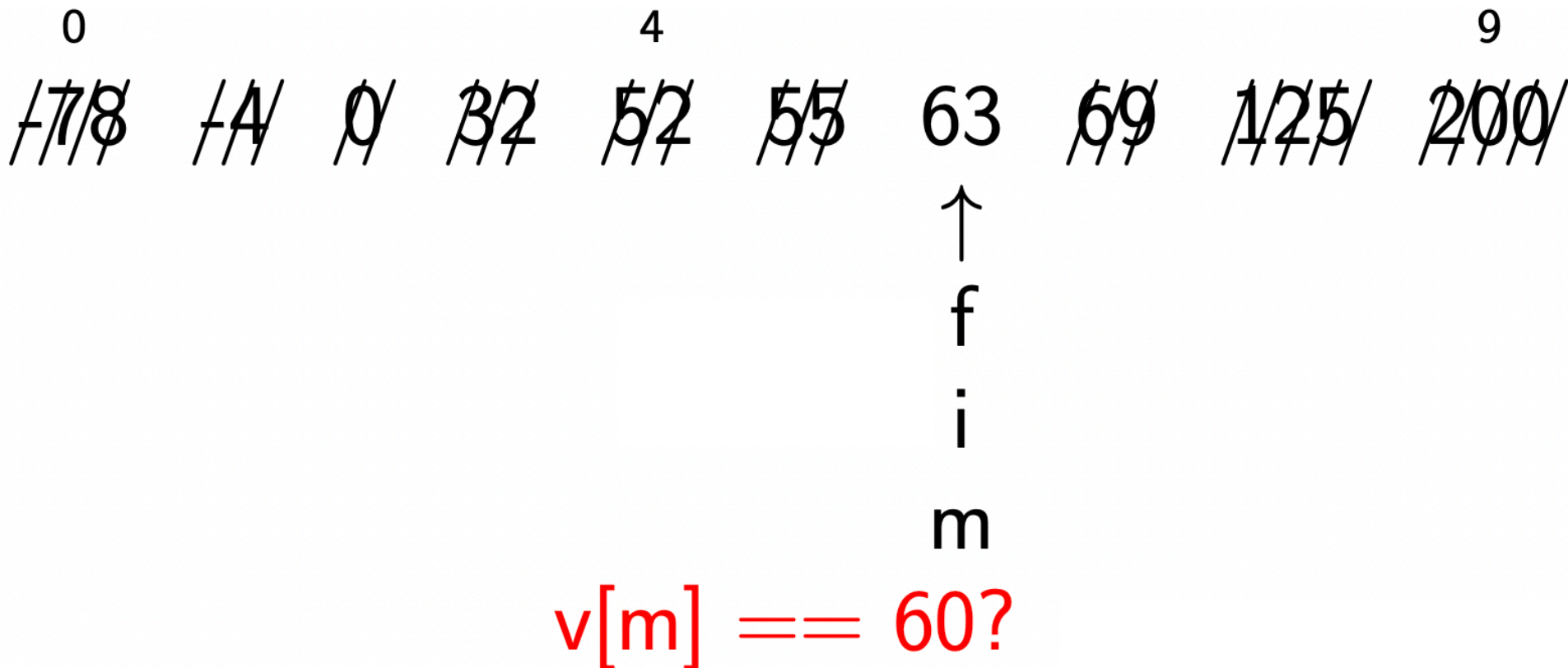
Busca Binária - Exemplo

- Buscando 60



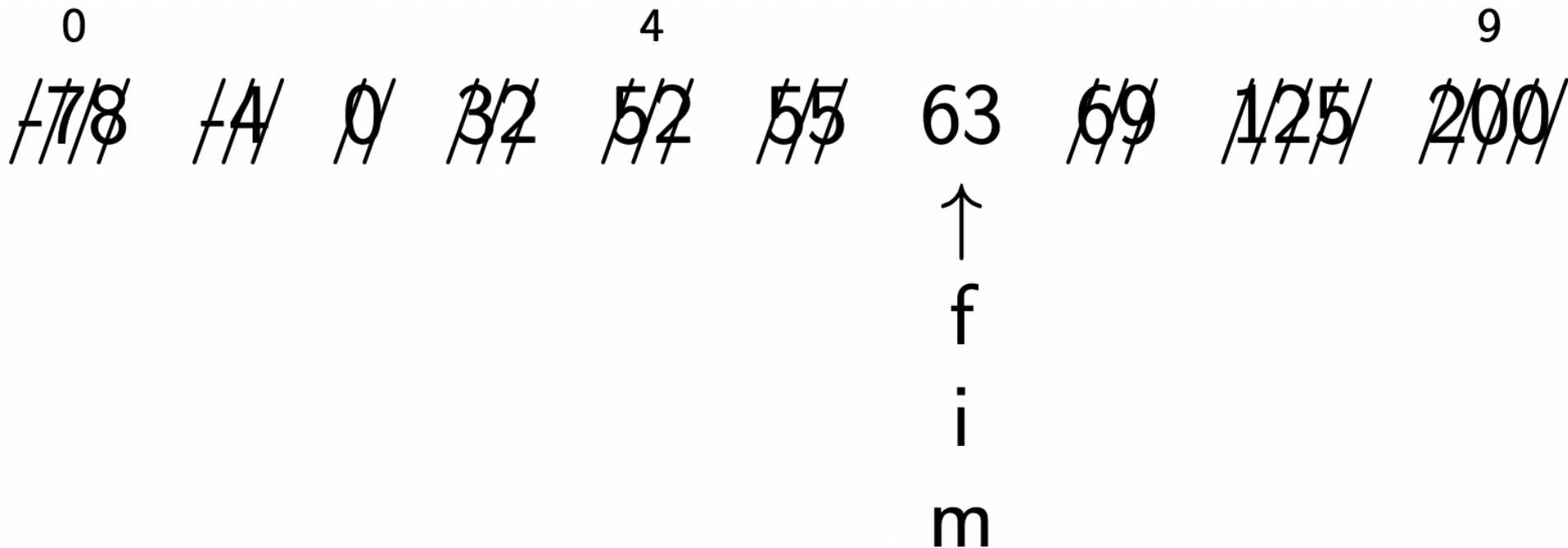
Busca Binária - Exemplo

- Buscando 60



Busca Binária - Exemplo

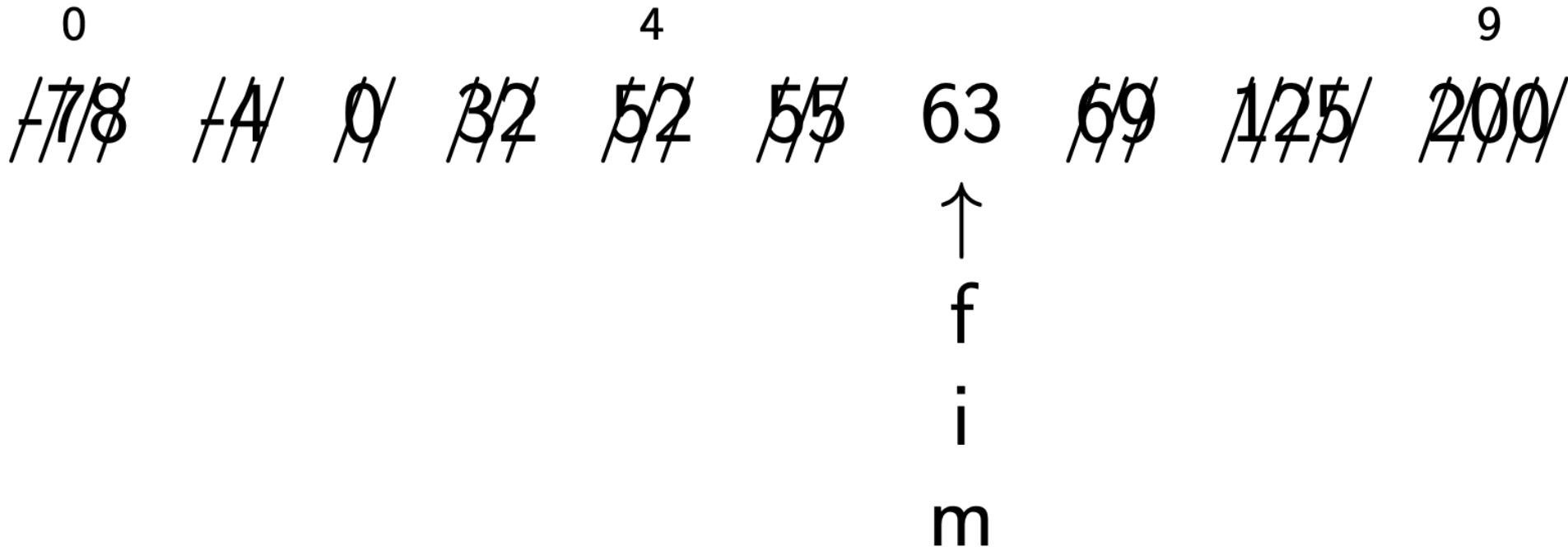
- Buscando 60



$v[m] == 60?$ Não. $v[m] < 60?$

Busca Binária - Exemplo

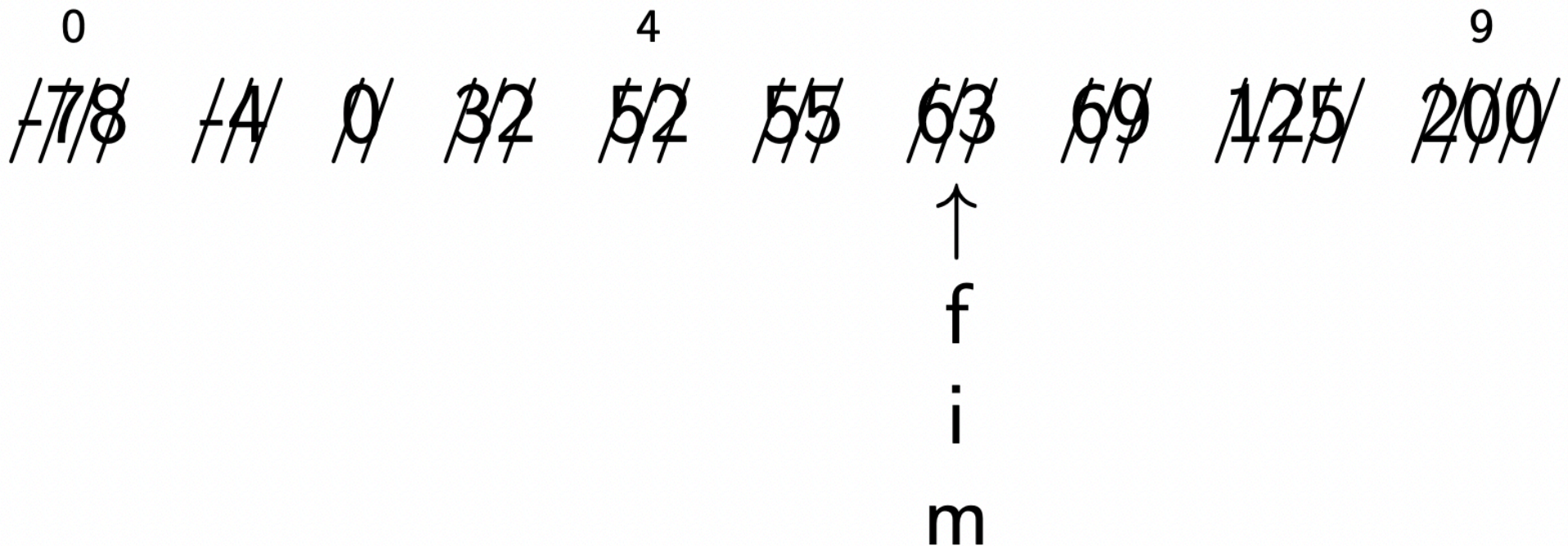
- Buscando 60



$v[m] == 60?$ Não. $v[m] < 60?$ Não

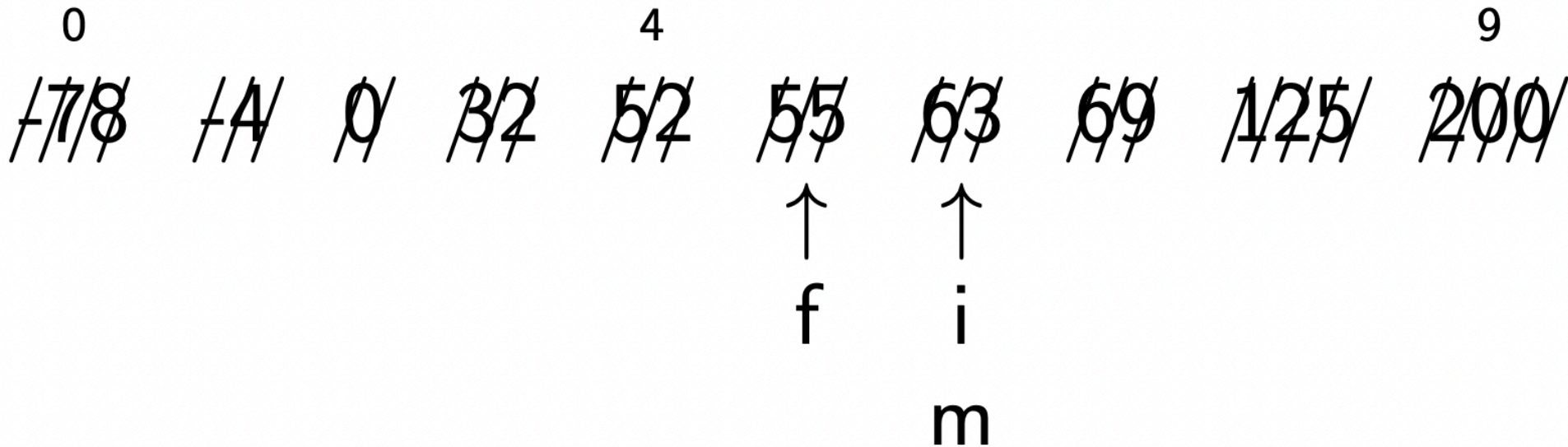
Busca Binária - Exemplo

- Buscando 60



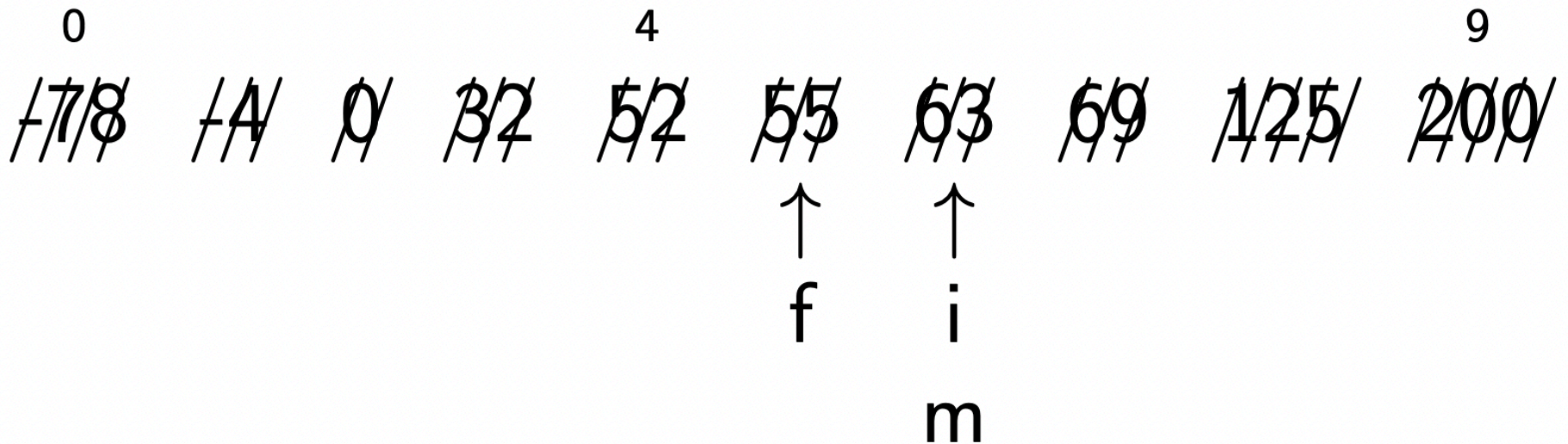
Busca Binária - Exemplo

- Buscando 60



Busca Binária - Exemplo

- Buscando 60



Inconsistência. O elemento não está no arranjo

Projeto de algoritmos

- Analisar um algoritmo significa prever os recursos de que ele vai precisar
 - **No tempo:** quanto tempo vai demorar para encontrar a solução do problema
 - **No espaço:** quanto de memória será necessário para encontrar a solução
- Projetar algoritmos implica analisar o seu comportamento conforme essas variáveis
- O estudo da eficiência de algoritmos (seja no tempo ou no espaço) é chamado **análise de algoritmos**

Complexidade

- Como medir o custo de um algoritmo?
- Como comparar o custo de vários algoritmos que resolvem um problema?

Complexidade

- 1. Medição direta do tempo de execução em um computador real
 - Problemas: depende do compilador; depende do *hardware*
 - Medidas de tempo podem ser influenciadas pela memória disponível

Complexidade

- 2. Medir com base em um **computador ideal**, em que cada instrução tem seu custo bem definido
 - Calcula-se o número de vezes que cada operação é realizada
 - Com uma tabela com o custo de cada tipo de operação, é possível calcular o custo total
 - Problemas: é algo bastante trabalhoso e o custo de cada operação varia de acordo com o *hardware*, compilador etc.

Complexidade

- 3. Calcula-se o **número de vezes que as operações mais significativas são executadas**, ignorando as demais operações
 - É a solução mais comumente usada e suficientemente precisa, especialmente quando queremos analisar a **complexidade assintótica**
 - Exemplo: na Busca ou na Ordenação $\Rightarrow$ número de comparações

Função de Complexidade

- Para medir o custo de execução de um algoritmo, definimos uma **função de custo** ou **complexidade** $f(n)$:
 - $f(n)$ é a medida do tempo ou espaço necessário para executar um algoritmo para uma entrada de tamanho n
 - Se for de tempo, então $f(n)$ é chamada de função de complexidade de tempo ou temporal do algoritmo
 - Se for da memória necessária (espaço) para executar o algoritmo, então $f(n)$ é a função de complexidade espacial

Função de Complexidade

- Se nada for dito, entende-se $f(n)$ como complexidade de tempo
 - Lembre que isso não representa o tempo diretamente, mas o número de vezes que determinada operação, considerada relevante, é executada
- Mas e isso funciona?
- Por que não condicionar apenas ao *hardware*, ou o tempo utilizado?
 - Porque não conseguiremos garantir as mesmas condições para toda e qualquer entrada

Função de Complexidade

- Normalmente, a medida de custo de execução depende do tamanho da entrada
 - Mas este não é o único fato que influencia o custo
- O tipo de entrada pode também influenciar o custo
 - Para um algoritmo de ordenação, é possível que menos tempo seja gasto se a entrada já estiver quase ordenada

Função de Complexidade

- **Cálculo da função de complexidade:**
 - Identifique o tempo de execução de cada comando no algoritmo
 - Selecione apenas os comandos relacionados com o tamanho da entrada
 - Ex: aqueles que irão ser executados um número de vezes proporcional a esse tamanho
 - A soma dos tempos de execução de cada um desses comandos corresponde ao tempo de execução do algoritmo

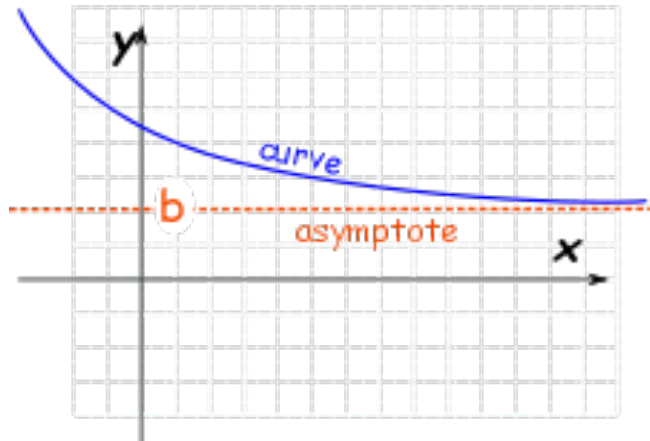
Crescimento Assintótico de Funções

- A escolha do algoritmo não é um problema crítico quando n é pequeno
 - O problema é quando n cresce
- Por isso, é usual analisar o comportamento das funções de custo quando n é bastante grande
 - Analisa-se o comportamento assintótico das funções de custo
 - Representa o limite do comportamento do custo quando n cresce

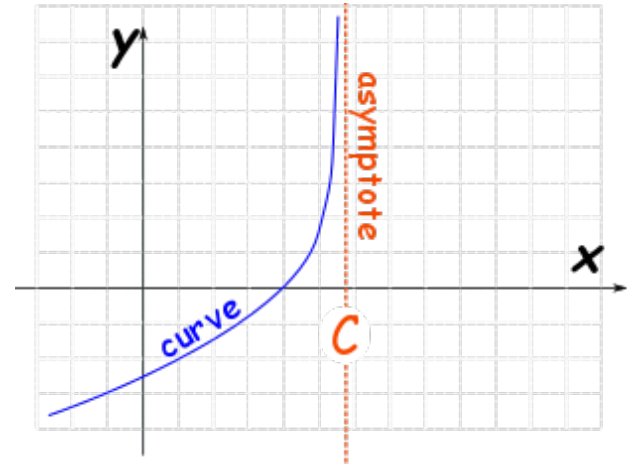
Crescimento Assintótico de Funções

- **O que significa “Comportamento Assintótico”?**
 - O comportamento assintótico descreve uma função ou expressão com um limite ou assíntota definidos
 - A função pode se aproximar desse limite, na medida em que a entrada muda, mas nunca o alcançará
- **Assíntota?**
 - A linha da qual uma curva se aproxima enquanto caminha ao infinito

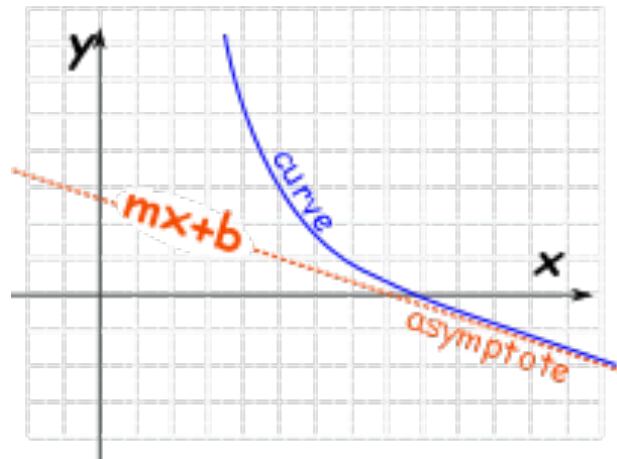
Crescimento Assintótico de Funções



Horizontais



Verticais



Oblíquas

Comportamento Assintótico

- Seja $f(n)$ a função de complexidade de um algoritmo A
 - O comportamento assintótico de $f(n)$ representa o limite do comportamento do custo (complexidade) de A quando n cresce sem restrições
 - Lembrando que a função de complexidade geralmente considera apenas algumas operações elementares, ou mesmo uma única operação elementar (ex: o número de comparações)
- A complexidade assintótica relata o crescimento assintótico das operações consideradas

Comportamento Assintótico

- Definir limites para o comportamento do algoritmo, identificando assim quando "o barco vai afundar"...
- Ex: 1 milhão (10^6) de operações por segundo

n						
Função de custo	10	20	30	40	50	60
n	0,00001s	0,00002s	0,00003s	0,00004s	0,00005s	0,00006s
n^2	0,0001s	0,0004s	0,0009s	0,0016s	0,0025s	0,0036s
n^3	0,001s	0,008s	0,027s	0,064s	0,125s	0,216s
n^5	0,1s	3,2s	24,3s	1,7min	5,2min	12,96min
2^n	0,001s	1,04s	17,9min	12,7dias	35,7 anos	366 séc.
3^n	0,059s	58min	6,5anos	3855séc.	10^8 séc.	10^{13} séc.

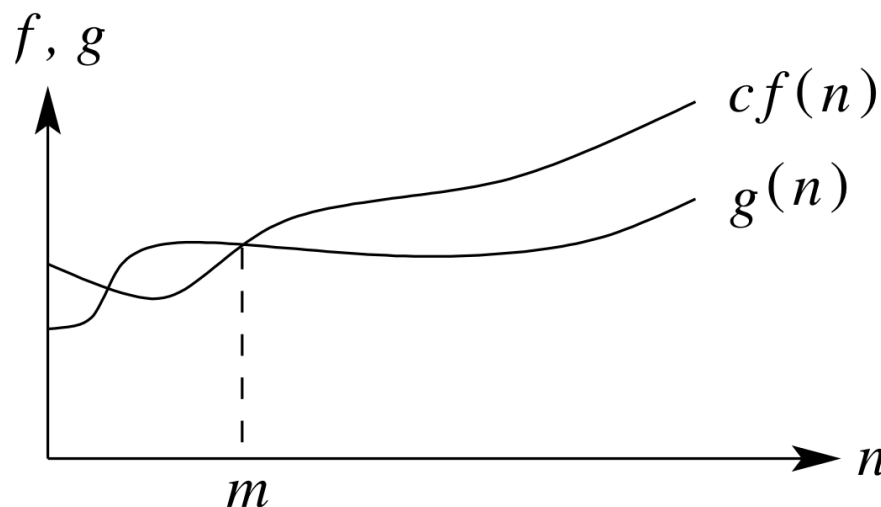
Comportamento Assintótico

- A definição de um limite nos dá uma caracterização simples da eficiência do algoritmo
 - Mesmo podendo determinar sua complexidade exata, o cálculo dessa precisão extra pode não valer o esforço
 - Para entradas grandes, as constantes multiplicativas e termos de menor ordem são dominados pelos efeitos do tamanho da entrada
- Nos permite também comparar o desempenho relativo de algoritmos alternativos
 - Nos diz qual será melhor quando a entrada cresce

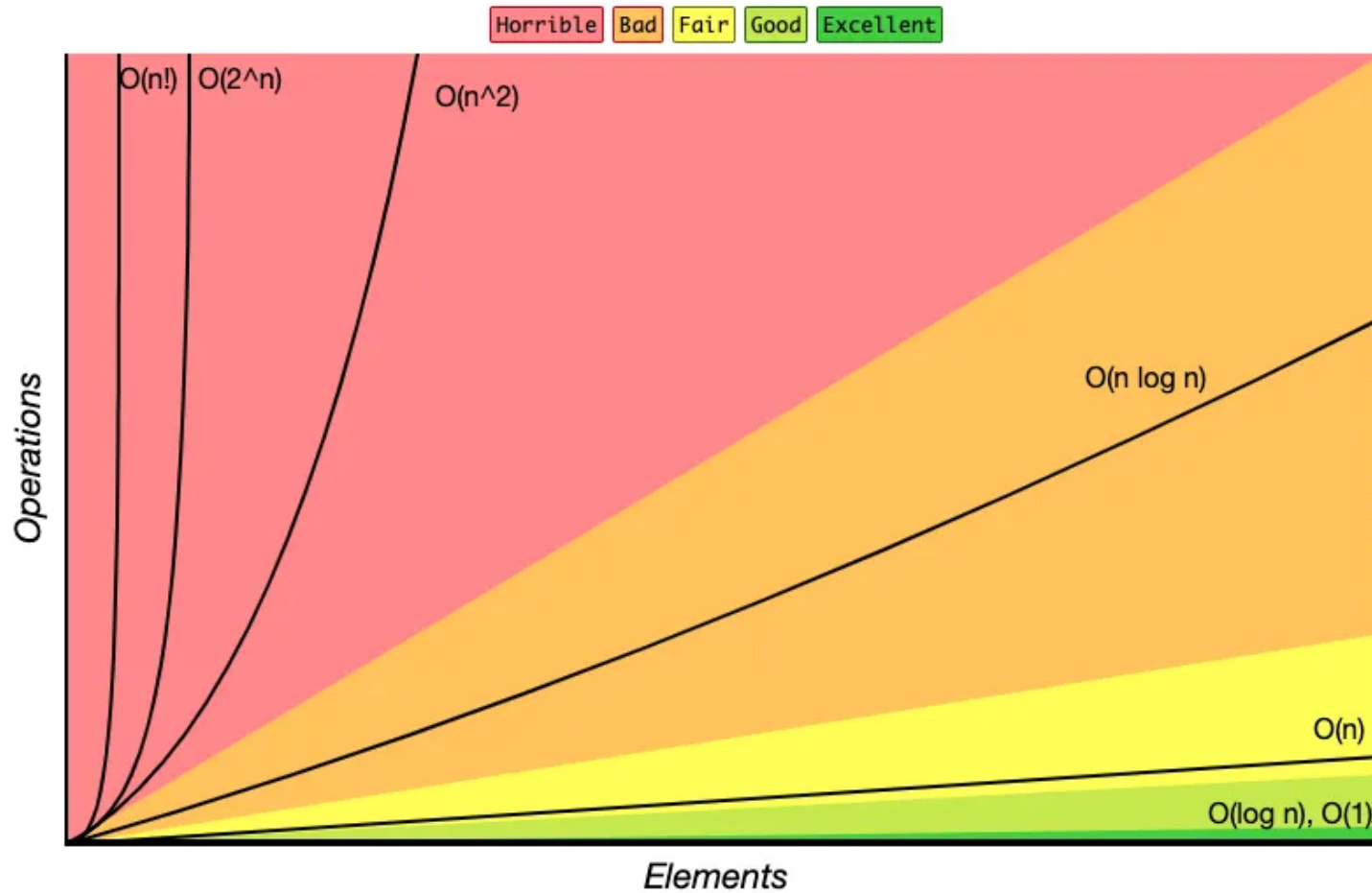
Notação O

- **Definição:** Uma função $g(n)$ é $O(f(n))$ se existirem constantes positivas c e m tais que $0 \leq g(n) \leq cf(n)$, para todo $n \geq m$

- Informalmente, dizemos que, se $g(n) \in O(f(n))$, então $g(n)$ cresce no máximo tão rapidamente quanto $f(n)$
- Trata-se então de um **limite assintótico superior** para $g(n)$



Big-O Complexity Chart



Fonte: <https://medium.com/nagoya-foundation/introdução-à-complexidade-de-algoritmos-4a9c237e4ecc>

Complexidade de algoritmos - Busca Sequencial

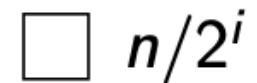
- Fazemos um máximo de n comparações
 - Onde n é o número de elementos do arranjo

Complexidade de algoritmos - Busca Binária

- E quantas fazemos com a busca binária?



...



Complexidade de algoritmos - Busca Binária

- Temos $i + 1$ comparações, sendo a última feita com o arranjo de tamanho 1
- A relação entre n e i é tal que, após i comparações, o arranjo terá $n/2^i$ elementos.
- Como no último nível há 1 elemento, então $n/2^i = 1 \Rightarrow n=2^i \Rightarrow \log_2(n)=i$

Assim temos $\log_2(n) + 1$ comparações

Complexidade de algoritmos - Busca

- Número de Comparações

Sequencial

- Melhor caso: O elemento é o primeiro
 - 1 comparação (arranjo ordenado ou não)
- Pior caso: O elemento não está no arranjo e é maior que todos
 - n comparações (arranjo ordenado ou não)

Binária

- Melhor caso: O elemento é o do meio
 - 1 comparação
- Pior caso: O elemento não está no arranjo
 - $\log_2(n) + 1$ comparações

Complexidade de algoritmos

- $\log_2(n) + 1 < n$ para $n \geq 3$
 - Para 1 e 2, $\log_2(n) + 1 = n$
- No **pior caso**, a busca binária é pelo menos tão boa quanto a sequencial, mas apenas para arranjos de tamanho mínimo
- Para os demais, ela é melhor

Complexidade de algoritmos

- Comparativo entre algoritmos de busca (considerando Comparações)

	Melhor	Pior
<i>Busca Sequencial</i>	$O(1)$	$O(n)$
<i>Busca Binária</i>	$O(1)$	$O(\log n)$